Path Names

1 Basic Idea

Definition

A **path name** is a name that identifies a file or directory by describing its location in a directory tree.

- Hierarchical file systems organize files as a tree of directories.
- A path name tells the operating system how to reach a file in that tree.
- Two common forms are absolute path names and relative path names.
- Absolute paths start from the root directory.
- Relative paths start from the current working directory.

2 Absolute Path Names

Absolute Path

An **absolute path name** gives the complete path from the root directory to the target file or directory.

- Absolute paths always start at the root directory.
- They are unique within a file-system tree.
- They do not depend on the process's current working directory.
- Example: `/usr/ast/mailbox`.
- This means: root → `usr` → `ast` → `mailbox`.

Rule

If the first character of a path is the path separator, the path is absolute.

3 Path Separators

System	Example absolute path
UNIX	`/usr/ast/mailbox`
Windows	`\usr\ast\mailbox`
MULTICS	`>usr>ast>mailbox`

- UNIX separates path components with `/`.

- Windows separates path components with backslash.
- MULTICS used `>`.
- The separator character differs, but the idea is the same.

4 Relative Path Names

Relative Path

A **relative path name** identifies a file relative to the process's current working directory.

- A relative path does not begin at the root directory.
- It is interpreted starting from the current working directory.
- If the current working directory is `/usr/ast`, then `mailbox` refers to `/usr/ast/mailbox`.
- Relative paths are often shorter and more convenient.
- Their meaning can change when the working directory changes.

5 Working Directory

Working Directory

The **working directory**, also called the current directory, is the directory used as the starting point for relative path names.

- Each process has its own working directory.
- Changing one process's working directory does not affect other processes.
- When the process exits, its working-directory change leaves no trace in the file system.
- A program can change its working directory when convenient.
- Library procedures should be careful: changing the working directory can surprise the rest of the program.

Library Warning

If a library procedure changes the working directory, it should change it back before returning.

6 Absolute vs. Relative Example

If the working directory is `/usr/ast`, these commands do the same thing:

```
cp /usr/ast/mailbox /usr/ast/mailbox.bak
cp mailbox mailbox.bak
```

- The first command uses absolute paths.
- The second command uses relative paths.
- Both copy `/usr/ast/mailbox` to `/usr/ast/mailbox.bak`.

7 When to Use Absolute Paths

- Use an absolute path when the program must access a specific file regardless of the working directory.
- Example: a spelling checker may need `/usr/lib/dictionary`.
- The spelling checker may be started from any directory.
- Therefore, using the full absolute path is safer.
- Absolute paths are less convenient to type but more predictable.

8 Changing Directory Instead

- A program that needs many files from one directory can change its working directory.
- Example: it can change to `/usr/lib`.
- Then it can open `dictionary` using a relative name.
- This works because the program explicitly knows where it is in the directory tree.

9 Dot and Dotdot

Special Directory Entries

Most hierarchical file systems provide `.` and `..` in every directory.

Entry	Meaning
`.`	The current directory.
`..`	The parent directory. In the root directory, it usually refers to the root itself.

- Dot is pronounced “dot”.
- Dotdot is pronounced “dot dot”.
- `..` lets a path move upward in the directory tree.
- `.` is useful when a command expects a directory name.

10 UNIX Directory Tree Example

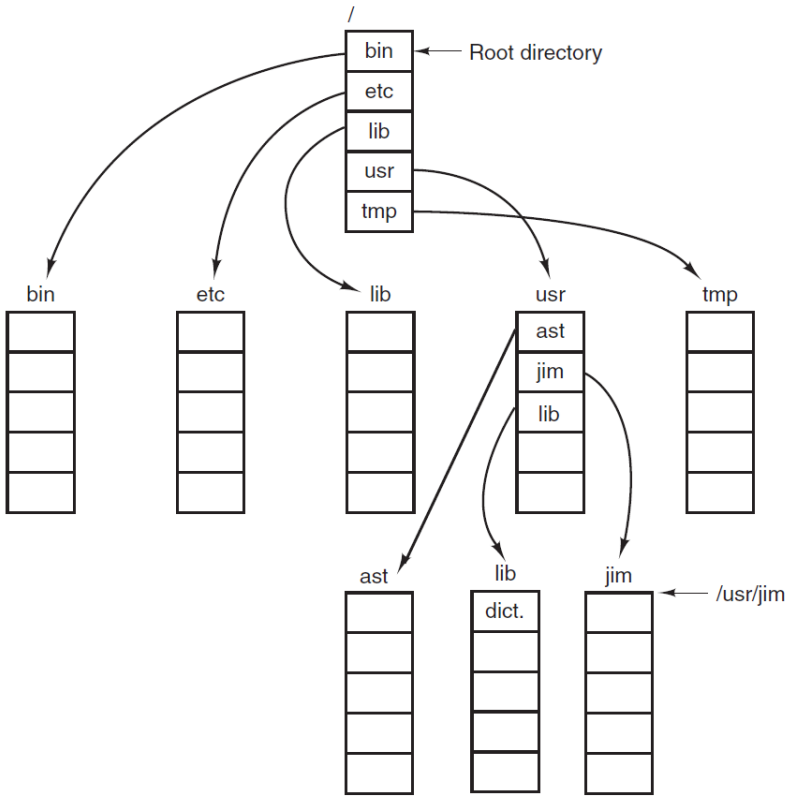

Figure 4-8. A UNIX directory tree.

Assume the current working directory is `/usr/ast`. The file `/usr/lib/dictionary` can be copied into the current directory using:

```
cp ../lib/dictionary .
```

- `..` moves from `/usr/ast` up to `/usr`.
- `lib` moves down into `/usr/lib`.
- `dictionary` selects the source file.
- `.` names the current directory, `/usr/ast`, as the destination.

11 Equivalent Copy Commands

These commands have the same effect when the working directory is `/usr/ast`:

```
cp ../lib/dictionary .
cp /usr/lib/dictionary .
cp /usr/lib/dictionary dictionary
cp /usr/lib/dictionary /usr/ast/dictionary
```

- The first command uses `..` and `.`

- The second uses an absolute source and current-directory destination.
- The third gives the destination file name explicitly.
- The fourth gives both source and destination as absolute paths.
- All identify the same source and destination.

12 Comparison

Feature	Absolute path	Relative path
Starting point	Root directory.	Current working directory.
Uniqueness	Unique independent of process location.	Meaning depends on working directory.
Convenience	Longer to type.	Usually shorter to type.
Best use	Programs needing a fixed file.	Interactive work inside a known directory.

Final Point

Absolute paths are complete and independent of the working directory; relative paths are shorter but depend on where the process currently is.